

Botox Markdown Suite

Minus

Systems Engineering

October 2026

Abstract – *A self-contained, single-binary command-line tool that converts standard Markdown documents into publication-grade LaTeX-style PDF documentation or presentation slide decks. The frontmatter defines document metadata and typography, while slide decks are delimited by standard horizontal rules. Built in Rust with an embedded in-process typesetting engine, requiring zero external runtime dependencies.*

Contents

1	Overview	1
2	Architecture	2
3	Installation	2
3.1	Using the Compiled Binary	2
3.2	Building from Source	2
4	Unified Command Usage	2
5	PDF Documentation	3
5.1	Frontmatter and Styling	3
5.2	Table of Contents Control	3
5.3	Command Line Options	3
6	Presentation Slides	4
6.1	Slide Syntax and Styling	4
7	Configuration and Defaults	4
7.1	Configuration Search Order	4
7.2	Configuration Schema	4
8	Mathematical Typography	5

/newpage

1 Overview

Botox provides a single, self-contained native binary (`botox`) to compile standard Markdown files into:

- **LaTeX-Grade PDF Documentation:** Typeset via an embedded, in-process Typst engine with authentic New Computer Modern typography, micro-typography, and TeX mathematical notation.

- **Presentation Slides:** Slide decks partitioned by horizontal rules (`---`) compiled directly into clean 16:9 presentation PDFs.

Mode detection is automatic based on the file frontmatter structure (or can be explicitly declared with `--pdf` or `--slides`).

This `README.md` file is itself an example document. You can compile it directly into a PDF using:

```
botox README.md -o README.pdf
```

2 Architecture

Unlike traditional document compilation pipelines that require a web of external runtimes:

- **Zero Runtime Dependencies:** No Node.js, no Python, no Pandoc, no Headless Chrome, and no gigabyte-scale TeX Live installations.
- **Embedded Typographic Engine:** The Typst layout engine and New Computer Modern font family (Roman, Bold, Italic, and Math) are compiled directly into the single executable.
- **Instant Compilation:** Typical compilation times range between 300ms and 1.5s.
- **Cross-Platform:** Compiles to a single standalone binary for Linux, macOS, and Windows.

3 Installation

3.1 Using the Compiled Binary

The executable can be placed anywhere on your system `PATH` :

```
# User-level binary location
cp target/release/botox ~/.local/bin/botox
```

3.2 Building from Source

To compile the standalone binary from source using the Rust toolchain:

```
cd botox
cargo build --release
strip target/release/botox
cp target/release/botox ~/.local/bin/botox
```

4 Unified Command Usage

The single command `botox` handles both documents and presentations:

```
# Compile a documentation PDF (auto-detected)
botox document.md -o document.pdf
```

```
# Compile a slide deck (auto-detected)
botox presentation.md -o presentation.pdf

# Generate document with an automatic Table of Contents
botox document.md --toc -o document.pdf

# Explicit mode declaration (optional)
botox pdf document.md -o document.pdf
botox slides presentation.md -o presentation.pdf
```

5 PDF Documentation

5.1 Frontmatter and Styling

The opening YAML block controls the metadata and styling of the document. You never need to construct a table of contents manually; it is generated dynamically with dotted leader lines and page numbers.

```
---
title: "Document Title"
author:
  - name: "Author Name"
    affiliation: "Organization or Lab"
date: "October 2026"
abstract: "Summary of the document."
mainfont: "New Computer Modern"
mathfont: "New Computer Modern Math"
fontsize: 11pt
papersize: a4
columns: 1
margin:
  x: 2.5cm
  y: 2.5cm
toc: true
---
```

5.2 Table of Contents Control

A Table of Contents is generated automatically when requested:

- In frontmatter: `toc: true` or `toc: false`
- Via command line: `--toc` or `--no-toc`
- In configuration: `toc: true` in `botox.yaml` to enable it by default across all documents

5.3 Command Line Options

- `-o, --output <file>`: Output PDF path (defaults to `<input>.pdf`).
- `--toc` / `--no-toc`: Enable or disable the automatic Table of Contents.
- `--author <name>`: Override author name.
- `--font <name>`: Override main typeface (e.g. `--font "Libertinus Serif"`).
- `-N, --number-sections`: Enable numbered headings (`1.`, `1.1.`, etc.).

- `--config <path>`: Path to a custom YAML configuration file.

6 Presentation Slides

6.1 Slide Syntax and Styling

Slides are divided by standard Markdown horizontal rules (`---`). The frontmatter block defines theme, background color, text color, and pagination:

```
---
marp: true
theme: default
paginate: true
backgroundColor: "#f8fafc"
color: "#0f172a"
---

# Slide 1 Title
Author Name

---

# Key Findings
- Sub-second single-binary compilation
- Embedded New Computer Modern fonts
- Zero runtime dependencies
```

Supported theme presets include `default`, `academic`, `dark`, and `nord`. Custom hex values can also be supplied via `backgroundColor` and `color`.

7 Configuration and Defaults

Default values for authors, fonts, margins, table of contents, and slide themes can be configured in a YAML file so they do not need to be specified in every document.

7.1 Configuration Search Order

- Path supplied via `--config <path>`
- `./botox.yaml` (project-level configuration in current working directory)
- `~/.config/botox/config.yaml` (user-level global configuration)

7.2 Configuration Schema

```
document:
  author: "Minus"
  affiliation: "Systems Engineering"
  fontsize: "11pt"
  mainfont: "New Computer Modern"
  mathfont: "New Computer Modern Math"
  papersize: "a4"
  columns: 1
```

```
margin:
  x: "2.5cm"
  y: "2.5cm"
section_numbering: false
toc: false

slides:
  theme: "default"
  paginate: true
```

Inspect active configuration settings:

```
botox config
```

8 Mathematical Typography

Mathematical formulas render with full TeX fidelity:

$$\mathcal{L}_G = \mathbb{E}_{x \sim p_{\text{data}}} [\log D(x)] + \mathbb{E}_{z \sim p_z} [\log(1 - D(G(z)))]$$

In-line formulas like $f(x) = \sum_{i=0}^n a_i x^i$ integrate seamlessly into text.